

QMWS Spring 2026: Introduction to Python for Data Analysis

Day One: Foundations

Gavin Klorfine & Deborah Laze

Section 1

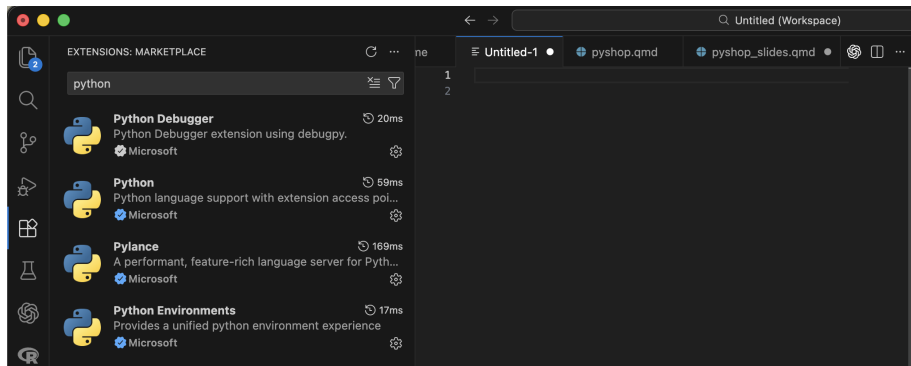
Installation

Visual Studio (VS) Code

- Installation link: <https://code.visualstudio.com/download>
- Integrated Development Environment (IDE)
- Can use it to code in many different languages
- Commonly used in both academia and industry

Visual Studio (VS) Code

- Once installed, go to the “Extensions” tab and search “Python”
- Install the below extensions:



- A tool for configuring Python **environments**
 - More often than not, you need to install **packages** that allow you to code more easily/efficiently
 - Allows for version control
 - Issues can arise if environments are not used, for example:
 - If you install two packages that conflict with one another
 - If you install a package that requires a specific version of Python
 - Anaconda allows these issues to be isolated to a specific environment, rather than affecting your whole Python installation
 - Especially useful if you have more than one project going on

- Installation link: <https://www.anaconda.com/download>
- The simplest way to use Anaconda is via the **Navigator**
- To install:
 - 1 Make an account
 - 2 Install the full distribution via the “Graphical Installer”

- Installation link: <https://www.anaconda.com/download>
- On Mac:



Choose Your Download

Windows

Mac

Linux

Anaconda Distribution

Complete package with 8,000+ libraries, Jupyter, JupyterLab, and Spyder IDE. Everything you need for data science.

↓ 64-Bit (Apple silicon)
Graphical Installer

↓ 64-Bit (Apple silicon)
Command Line Installer

Miniconda

Minimal installer with just Python, Conda, and essential dependencies. Install only what you need.

↓ 64-Bit (Apple silicon)
Graphical Installer

↓ 64-Bit (Apple silicon)
Command Line Installer

↓ 64-Bit (Intel chip) Graphical
Installer

- Installation link: <https://www.anaconda.com/download>
- On Windows:



Choose Your Download

Windows

Mac

Linux

Anaconda Distribution

Complete package with 8,000+ libraries, Jupyter, JupyterLab, and Spyder IDE. Everything you need for data science.

 [Windows 64-Bit Graphical Installer](#)

Miniconda

Minimal installer with just Python, Conda, and essential dependencies. Install only what you need.

 [Windows 64-Bit Graphical Installer](#)

Section 2

Setting Up a Conda Environment

Section 3

The Basics

- In programming, a variable is a space in computer memory where values may be stored

Defining and Printing Variables

```
x = 67
```

```
print(x)
```

```
67
```

Arithmetic

```
a = 1
```

```
b = 2
```

```
print(a + b)      # * for multiplication, / for division
```

```
3
```

Note

- Use # to make a **comment**

Updating Variables

```
x = 10
```

```
x = 20
```

```
x = 15
```

```
print (x)
```

```
15
```

Variable Types

- ❶ `int` (integer; “whole number”)
 - less memory
 - 1 is an `int`
- ❷ `float` (real; “decimal number”)
 - 1.2 is a `float`
 - 1.0 is a `float`
- ❸ `str` (string; characters/words with no numerical meaning)
 - “Hello world!” is a string
 - “416-676-6767” is also a string
- ❹ `bool` (boolean; can be `True` or `False`; more on this later)
- ❺ `None`
 - A special type of variable that essentially means “nothing meaningful is stored”
 - Useful as a placeholder

Variable Types

Checking Type

```
x = 416
print(x, type(x))    # type() to check type

416 <class 'int'>
```

Converting Type

```
x = 416
x = str(x)           # str() to convert to string
print(x, type(x))

416 <class 'str'>
```

- Can also use `int()` and `float()` to convert to respective type

A Brief Note

- `print()`, `type()`, `str()`, etc. are all examples of **functions**
 - These are essentially pre-made chunks of code for you to use
 - More on this later!

What will the below code output?

```
x = "Variable"

print(type(x))
print(x)
```

Section 4

More Advanced Variable Types

- Multiple values can be stored in one variable. A list is one way to do this:

Defining Lists

```
x = [0, 1, 2, 3, 4]      # Define a list; square brackets  
print(type(x))
```

```
<class 'list'>
```

Lists: Indexing

```
x = [0,1,2,3,4]
```

- To extract one or more values from a list, you **index** the list
- Python uses **zero-based indexing**
 - This means that the first element of a list is index 0, the second element is index 1, ...
 - Element x is index $x - 1$

Indexing Lists

```
x = [0, 1, 2, 3, 4]
```

```
y = 2
```

```
print(x[0])      # Index first element of a list
```

```
print(x[-1])     # Index last element
```

```
print(x[y])      # Index yth element
```

```
0
```

```
4
```

```
2
```

Lists: Indexing

- You can also take more than one element, or a **slice**, of the list

Slicing Lists

```
x = [0, 1, 2, 3, 4]
```

```
print(x[0:2])    # first and second (but not third) elements
```

```
[0, 1]
```

! Important

- [inclusive:exclusive]
 - The index on the *left-hand* side of the slice is *included*, whereas the index on the *right-hand* side of the slice is *excluded*

Slicing Lists, continued

```
x = [0, 1, 2, 3, 4]
```

```
print(x[0:3])    # first three elements
```

```
print(x[0:1])    # a (silly) way to index just the first element
```

```
[0, 1, 2]
```

```
[0]
```

Lists: Length

- To find the length of a list:

len() function

```
x = [0, 1, 2, 3, 4]
```

```
print(len(x))    # The length of list x
```

```
# Another way to find the last element (not so silly!)
```

```
print(x[len(x) - 1])
```

5

4

Lists: Methods

- Lists have certain functions that can be applied to them, called **methods**
 - Methods are more general than this (e.g., string methods; more on these in a little bit!)

```
.append()
```

```
x = [0,1,2,3,4]
```

```
x.append(5)
```

```
print(x)
```

```
[0, 1, 2, 3, 4, 5]
```

- Adds an entry to a list

`.extend()`

```
x = [0,1,2,3,4]
```

```
x.extend([6,7,8,9])
```

```
print(x)
```

```
[0, 1, 2, 3, 4, 6, 7, 8, 9]
```

- Extend a list by “glueing” it to another list

Difference Between `.append()` and `.extend()`

```
x = [0,1,2,4]
```

```
y = [5,6,7,8]
```

```
x.append(y)
```

```
print(x)
```

```
x = [0,1,2,4]    # Redefine x
```

```
x.extend(y)
```

```
print(x)
```

```
[0, 1, 2, 4, [5, 6, 7, 8]]
```

```
[0, 1, 2, 4, 5, 6, 7, 8]
```

Lists: Changing Values

Changing a Value in a List

```
x = [0, 1, 2, 3, 4]
```

```
x[0] = 9
```

```
print(x)
```

```
[9, 1, 2, 3, 4]
```

Tuples (Briefly)

- Like lists, tuples contain multiple values
- Unlike lists, tuples are **immutable**; they cannot be changed once created.
 - Lists are *mutable*

Defining a Tuple

```
x = ("a", "tuple")  # Round brackets
```

```
print(x[0], x[1])
```

```
print(type(x))
```

```
a tuple
```

```
<class 'tuple'>
```

Tuples (Briefly)

Changing Values... ERROR!

```
x = ("a", "tuple")
```

```
x[0] = "Not a"
```

TypeError: 'tuple' object does not support item assignment

TypeError

Traceback (most recent call last):

Cell In[100], line 3

```
1 x = ("a", "tuple")
```

```
----> 3 x[0] = "Not a"
```

TypeError: 'tuple' object does not support item assignment

Tuples (Briefly)

- Tuples have special properties, one of which is below:

Unpacking Tuples

```
x, y = (6, 7)
```

```
print(x)
```

```
print(y)
```

```
6
```

```
7
```

Section 5

Strings

Strings: Basics

- Strings are a lot like lists in that they are **iterable**
 - For now, think of these (lists, strings) as a sequence of values

Strings are Like Lists!

```
cringe = "Hello world!"
```

```
print(cringe[0:5])  
print(len(cringe))
```

```
Hello  
12
```

Strings: Basics

- Strings are also kind of like tuples in that they are **immutable**

String Immutability

```
cringe = "Hello world!"  
cringe[0:5] = "Goodbye"      # Error
```

TypeError: 'str' object does not support item assignment

TypeError Traceback (most recent call last):

Cell In[103], line 2

```
1 cringe = "Hello world!"  
----> 2 cringe[0:5] = "Goodbye"      # Error
```

TypeError: 'str' object does not support item assignment

- We can turn to string **methods** to “modify” strings when needed (they just create a new string “behind the scenes”)

```
.replace()
```

```
cringe = "Hello world!"
```

```
print(cringe.replace("Hello", "Goodbye"))
```

```
Goodbye world!
```

`.split()`

```
meme_list = "baby gronk, 67, skibidi toilet, the rizzler"
```

```
print(meme_list.split(", ")) # Split on ", "; returns a list
```

```
print(meme_list.split(", ")[0]) # Index the list
```

```
['baby gronk', '67', 'skibidi toilet', 'the rizzler']
```

```
baby gronk
```

How might you index “the rizzler”?

Strings: Methods

How might you index “the rizzler”?

```
meme_list = "baby gronk, 67, skibidi toilet, the rizzler"

print(meme_list.split(", ")[3])
print(meme_list.split(", ")[-1])
print(meme_list.split(", ")[len(meme_list.split(", ")) - 1])

the rizzler
the rizzler
the rizzler
```

Strings: f-strings

- If you wanted to include variables within text, format strings as per the following:

f-string Example

```
x = 3.14159
```

```
print(f'Approximate value of pi is {x}')
```

```
print(f'To three decimal places is {x:.3f}')
```

```
Approximate value of pi is 3.14159
```

```
To three decimal places is 3.142
```

Strings: f-strings

- The string (surrounded by either single `' '` or double `" "` quotations) is preceded by `f`
- The variable is surrounded by curly braces `{}` within the string
- Formatting is applied to the variable *within the curly braces*
 - `A :` signals that the proceeding formatting code should be applied to the variable

f-string Formatting Basics

- `x:.3f`
 - Format variable `x`
 - Round to 3 decimal places
 - `f` means float

Section 6

Booleans

Booleans

- A boolean variable (type `bool`) can be either `True` (1) or `False` (0).

Booleans

```
x = True
y = 10
print(type(x), type(y))

print (y > 5)
print (y < 5)
print(type(y < 5))

<class 'bool'> <class 'int'>
True
False
<class 'bool'>
```

Comparison Operators

- > greater than
- < less than
- >= greater than or equal to
- <= less than or equal to
- == equal to
- != not equal to

Logical Operators

- and
- or
- not

Section 7

Conditional Statements

Conditional Statements

- If thing1 happens, do x
- If thing2 happens instead, do y
- Otherwise, do z

Conditional Statements

- If thing1 happens, do x
- If thing2 happens instead, do y
- Otherwise, do z

Basic if/elif/else

```
x = 10 # Value presumably unknown  
y = 20
```

```
if x == 4:  
    print("x is 4")  
elif x > y:  
    print("x is bigger than y")  
else:  
    print("none of the above")
```

none of the above

Conditional Statements

Structure of Conditionals

```
if x == 4:  
    print("x is 4")
```

- Begins with `if/elif` ("else if")/else
- Boolean expression/value
- Colon :
- Indented code underneath that runs if condition is met

Section 8

Loops

while Loops

- Loops are used to run a chunk of code repeatedly
 - Often a set number of times
- This is best shown and unpacked through examples:

while Loops

while Loop Basics

```
count = 1    # Initialize counter
```

```
while count <= 5:  
    print(count)  
    count = count + 1    # Increment counter
```

```
1  
2  
3  
4  
5
```

while Loops

Iterating through a list

```
x = [0,1,2,3]
```

```
count = 0
```

```
while count <= len(x) - 1:
```

```
    print(x[count])
```

```
    count += 1      # Same as count = count + 1
```

0

1

2

3

while Loops: the break Statement

- The break statement is used to exit a loop

break

```
x = 0

while True:      # Infinite loop...
    if x > 10:
        break    # Until you break!
    x += 1

print(x)
```

11

for Loops

- Often used with the `range()` function to loop over code a set number of times

for Loops, `range()`

```
for i in range(3):  
    print(i)
```

0

1

2

for Loops

Iterating Over Entries in a List

```
x = [10,20,30,67]
```

```
for i in x:  
    print(i)
```

10

20

30

67

for Loops: enumerate()

- Use `enumerate()` and a second indexing variable (usually `i` or `j`) if you want to keep variable position

```
enumerate()
```

```
x = [10,20,30,67]
```

```
for i, j in enumerate(x):  
    print(i, j)
```

```
0 10
```

```
1 20
```

```
2 30
```

```
3 67
```

for Loops: Nesting

- Loops (and conditionals) may be nested inside one another (like Russian dolls / Matryoshka)

Nesting

```
x = ["big", "small"]  
y = ["cat", "dog"]
```

```
for i in x:  
    for j in y:  
        print(i, j)
```

```
big cat  
big dog  
small cat  
small dog
```

Section 9

Importing Packages, Defining Functions

Importing Packages

- Some packages are installed with Python by default, and no extra work is required to use them

```
import math
```

```
import math
```

```
print(f'pi = {math.pi:.3f}')
```

```
pi = 3.142
```

Importing Packages

- We can rename packages to make their usage more concise

Renaming Packages

```
import math as m

print(f'pi = {m.pi:.3f}')

pi = 3.142
```

Importing Packages

- Some require installation. This is most easily (and safely) done through **anaconda**

Importing Packages

```
import numpy as np
```

```
import numpy as np          # Common convention
```

```
# Take a sample of 3 values from N(0, 1)
```

```
print(np.random.normal(loc = 0, scale = 1, size = 3))
```

```
[ 1.99505889 -1.08469598 -0.24511993]
```

Note

- $\text{loc} = \mu$, $\text{scale} = \sigma$

Defining Functions

- Sometimes you repeat the same code over and over again
 - When this happens, it is useful to collapse the repeated code into a function
 - Otherwise, code becomes long and messy due to copy-pasting

Defining Functions

Defining Basic Functions

```
def addition(a, b):  
    return a + b          # What the functions spits out  
  
print(addition(a = 1, b = 5)) # Call function  
  
def subtraction(a, b = 1):    # Give b default value of 1  
    return a - b  
  
print(subtraction(a = 5))
```

6

4

Defining Functions

Function Example

```
x = [0,1,2,3,4,5,6,7]
```

```
def qm_mean(things):
```

```
    sum = 0
```

```
    for i in things:
```

```
        sum += i
```

```
    return sum / len(x)
```

```
qm_mean(x)
```

3.5